

Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 2, Магистратура, группа 12-25РПм
Херсонская область - 2026

Дисциплина:
«Управление конфигурацией и сопровождение программного обеспечения»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 2-го курса

Проверил:
Гуцол Д.В.

ПРАКТИЧЕСКАЯ РАБОТА 4

«Согласование изменений и проверка версии»

I. ЦЕЛЬ И ЗАДАЧИ

1.1. Цель

Целью данной практической работы является изучение процесса согласования независимых изменений в Git, выявление причины отклонения push при расхождении удалённой и локальной истории, выполнение безопасного слияния изменений, публикация проверенной версии проекта и восстановление её истории из переносимого Git bundle.

1.2. Задачи

- Подготовить локальный учебный стенд Git с центральным bare-репозиторием team.git и двумя рабочими копиями alice и bob.
- Проверить исходное состояние обеих рабочих копий.
- Выполнить согласованное изменение конфигурации и документации в копии alice.
- Проверить корректность изменения с помощью check.sh.
- Зафиксировать изменение коммитом и отправить его в удалённый репозиторий.
- Выполнить независимое изменение в копии bob.
- Зафиксировать независимое изменение отдельным коммитом.
- Получить и сохранить сообщение об отклонённом push из-за non-fast-forward.
- Объяснить причину отклонения push.
- Получить изменения удалённого репозитория с помощью git fetch.
- Проанализировать различия локальной и удалённой истории.
- Выполнить автоматическое слияние двух независимых изменений.
- Повторно проверить конфигурацию после merge.
- Создать аннотированный тег v0.3.0 для проверенной версии.
- Опубликовать main и тег v0.3.0 в team.git.
- Получить проверенную версию в копии alice.
- Сравнить хеши коммитов в alice, bob, team.git и теге.
- Создать переносимый Git bundle и release.zip.
- Клонировать историю из bundle в каталог restored.
- Проверить восстановленную версию с помощью check.sh.
- Сформировать итоговый отчёт и архив результатов.

II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

2.1. Git-репозиторий и рабочие копии

В практической работе используется центральный bare-репозиторий team.git, который моделирует общий удалённый репозиторий. Рабочие копии alice и bob являются независимыми локальными клонами. Каждая копия имеет собственную локальную историю и может создавать коммиты независимо от другой.

2.2. Fast-forward и non-fast-forward

Fast-forward возможен, когда удалённая ветка содержит линейное продолжение локальной истории и для обновления не требуется создавать отдельный merge-коммит. Если локальная и удалённая ветки имеют независимые коммиты, обычный push не может безопасно переместить указатель удалённой ветки без потери чужого изменения. В таком случае сервер отклоняет push с сообщением non-fast-forward или fetch first.

2.3. Независимые изменения

В работе роли alice и bob выполняют разные изменения. Alice изменяет config.txt и README.txt, устанавливая интервал 30 секунд и документируя это значение. Bob независимо создаёт CHANGELOG.txt с информацией о версии v0.3.0. Поскольку изменения затрагивают разные файлы, они могут быть объединены автоматически.

2.4. Причина отклонения push

После успешного push Alice удалённая ветка main содержит её новый коммит, которого нет в локальной ветке Bob. Bob создаёт собственный коммит поверх старого состояния и пытается отправить его напрямую. Git отклоняет такой push, поскольку удалённая ветка уже содержит историю, отсутствующую локально. Это защищает опубликованный коммит Alice от несанкционированной перезаписи.

2.5. git fetch и получение истории

Команда git fetch origin получает актуальное состояние удалённого репозитория, не изменяя непосредственно локальную рабочую ветку. После fetch Bob получает ссылку origin/main, которая позволяет сравнить его локальную историю с удалённой и подготовить безопасное слияние.

2.6. Слияние изменений

Команда git merge origin/main объединяет локальный коммит Bob с ранее опубликованным коммитом Alice. В рассматриваемом сценарии изменения находятся в разных файлах, поэтому Git выполняет автоматическое слияние без текстового конфликта. Результатом становится merge-коммит с двумя родителями.

2.7. Проверка версии перед публикацией

После слияния необходимо повторно выполнить check.sh. Это позволяет убедиться, что изменение конфигурации и соответствующая документация согласованы. Успешное прохождение проверки является основанием для публикации проверенной версии.

2.8. Аннотированный тег

Тег v0.3.0 используется для обозначения проверенного состояния проекта. Аннотированный тег содержит собственный объект Git и сообщение Verified shared configuration. Публикация тега позволяет однозначно ссылаться на выпущенную версию независимо от последующих изменений ветки main.

2.9. Git bundle и восстановление истории

Git bundle позволяет упаковать историю репозитория в переносимый файл. В работе созданный bundle используется для получения новой копии restored. Если после клонирования из bundle тег v0.3.0 указывает на тот же коммит, а check.sh завершается успешно, можно сделать вывод о переносимости проверенной истории.

III. МЕТОДЫ И ПОДХОДЫ К ВЫПОЛНЕНИЮ РАБОТЫ

3.1. Подготовка рабочего окружения

- Работа выполнялась в Windows 11 с использованием Git Bash.
- Для эксперимента использовалась отдельная рабочая папка git-remote-p2.
- После запуска setup.sh были созданы team.git, alice и bob.
- Обе рабочие копии находились на ветке main.
- Исходным состоянием проекта был принят тег v0.2.0.
- В каждой рабочей копии имелся один подготовленный коммит.

3.2. Исходная проверка репозитория

- Команда pwd использовалась для контроля текущей директории.
- Команда git --version подтвердила наличие Git.
- Команда bash setup.sh подготовила учебный стенд.
- Команды git -C alice status -sb и git -C bob status -sb подтвердили чистое состояние рабочих копий.
- Обе копии находились на ветке main и были синхронизированы с исходной версией.

3.3. Изменение в копии Alice

- В копии alice был настроен автор коммитов Student Alice.
- В config.txt установлено значение interval=30.
- В README.txt добавлено описание интервала 30 секунд.
- Запущена проверка bash check.sh.
- Проверка завершилась результатом PASS: interval=30; documentation matches.
- Изменения были зафиксированы коммитом с сообщением Set and document interval 30.
- Коммит успешно отправлен в origin/main.

3.4. Независимое изменение в копии Bob

- В копии bob был настроен автор коммитов Student Bob.
- До получения изменений Alice команда `cat config.txt` показала исходное значение `interval=60`.
- В bob был создан `CHANGELOG.txt`.
- В файл были записаны сведения о версии `v0.3.0`.
- Изменение было зафиксировано отдельным коммитом `Add release notes`.

3.5. Фиксация отклонённого push

- Bob выполнил `git push origin main`.
- Push был отклонён удалённым репозиторием.
- Сообщение Git сохранено в `evidence/rejected-push.txt`.
- В сообщении присутствовали признаки `rejected` и `non-fast-forward`.
- Повторный push с `--force` и предварительный pull до этапа согласования не выполнялись.
- Причиной отказа являлось наличие нового коммита Alice в `origin/main`.

3.6. Получение и анализ удалённой истории

- В bob выполнена команда `git fetch origin`.
- Команда `git log --oneline --graph --all --decorate` использовалась для визуального анализа истории.
- Команда `git log --oneline main..origin/main` показала коммит Alice, отсутствующий локально.
- Команда `git log --oneline origin/main..main` показала собственный коммит Bob, отсутствующий в `origin/main`.
- Таким образом, локальная и удалённая истории имели независимые изменения.

3.7. Слияние историй

- Выполнена команда `git merge --no-edit origin/main`.
- Git автоматически объединил изменения Alice и Bob.
- Текстовый конфликт не возник, поскольку изменения затрагивали разные файлы.
- После `merge` была выполнена `bash check.sh`.
- Проверка завершилась результатом `PASS` для `interval=30`.
- `CHANGELOG.txt` сохранился в объединённой версии.

3.8. Проверка merge-коммита

- Команда `git rev-list --parents -n 1 HEAD` использовалась для определения родителей merge-коммита.
- Результат показал хеш текущего merge-коммита и два родительских хеша.
- Это подтвердило, что две независимые линии разработки были объединены в одну историю.
- Рабочее дерево после `merge` находилось в согласованном состоянии.

3.9. Публикация проверенной версии

- Создан аннотированный тег `v0.3.0` с сообщением `Verified shared configuration`.
- Команда `git push origin main v0.3.0` отправила обновлённую ветку и тег в `team.git`.
- Команда `git ls-remote --tags origin` подтвердила наличие опубликованного тега.
- Проверенная версия была зафиксирована в удалённом репозитории.

3.10. Получение общей версии Alice

- В `alice` выполнена команда `git pull --ff-only origin main`.
- Получение прошло без необходимости создавать дополнительное слияние.
- После этого выполнен `git fetch origin --tags`.
- Проверка `bash check.sh` завершилась успешно.
- Хеши `HEAD` в `alice`, `bob` и `main` репозитория `team.git` совпали.
- Хеш коммита, на который указывает `v0.3.0`, также совпал с `HEAD`.

3.11. Упаковка и восстановление

- В корневой папке стенда выполнен `bash pack.sh`.
- В каталоге `submission` были сформированы `work.bundle`, `history.txt`, `state.txt`, `report.txt`, `evidence` и `release.zip`.
- Из `work.bundle` создана новая рабочая копия `restored`.
- Команда `git -C restored status -sb` проверила состояние восстановленной копии.
- Команда `git -C restored rev-parse "v0.3.0^{commit}"` подтвердила наличие тега и его привязку к проверенному коммиту.
- В `restored` выполнен `bash check.sh`.
- Проверка завершилась результатом `PASS`.

IV. ПРАКТИЧЕСКОЕ ВЫПОЛНЕНИЕ И АНАЛИТИЧЕСКОЕ ОСМЫСЛЕНИЕ

4.1. Исходное состояние

- После запуска `setup.sh` были получены три Git-репозитория: `team.git`, `alice` и `bob`.
- Alice и Bob начинали работу с одной исходной версии.
- Исходная версия проекта была обозначена тегом `v0.2.0`.
- Каждая рабочая копия содержала один подготовленный коммит.
- Рабочие деревья находились в чистом состоянии.

4.2. Изменение Alice

- В Alice параметр конфигурации был изменён с `interval=60` на `interval=30`.
- `README.txt` был синхронизирован с новым значением.
- Проверка вернула `PASS: interval=30; documentation matches`.
- Коммит получил условный хеш `7a31c2d`.
- После `push` этот коммит стал частью `origin/main`.

4.3. Независимое изменение Bob

- Bob продолжил работу со старой локальной версией конфигурации, содержащей `interval=60`.
- Был создан `CHANGELOG.txt`.
- Коммит получил условный хеш `c84f91e`.
- Изменение не зависело от файлов, изменённых Alice.

4.4. Отклонение push

- При попытке Bob выполнить push удалённый репозиторий обнаружил, что `origin/main` содержит коммит Alice.
- Локальная ветка Bob содержала собственный коммит поверх старого состояния.
- Git отклонил push с сообщением `non-fast-forward`.
- Условный фрагмент сообщения: `! [rejected] main -> main (non-fast-forward)`.
- В `evidence/rejected-push.txt` было сохранено полное сообщение команды.
- Отклонение является штатным механизмом защиты истории, а не ошибкой Git.

4.5. Сравнение историй

- `origin/main` содержал коммит `7a31c2d` Alice.
- Локальная `main` Bob содержала коммит `c84f91e` Bob.
- Команда `main..origin/main` показывала изменение Alice.
- Команда `origin/main..main` показывала изменение Bob.
- Таким образом, ветви разошлись после общего предка.

4.6. Автоматическое слияние

- Команда `git merge --no-edit origin/main` создала новый merge-коммит.
- Условный хеш merge-коммита: `f52d8a1`.
- Родителями merge-коммита стали `7a31c2d` и `c84f91e`.
- Конфликтов не возникло, поскольку Alice изменяла `config.txt` и `README.txt`, а Bob добавлял `CHANGELOG.txt`.
- После merge проект содержал изменения обеих ролей.

4.7. Проверка объединённой версии

- После merge команда `bash check.sh` вернула PASS.
- Значение `interval` составило 30.
- Документация соответствовала значению конфигурации.
- CHANGELOG.txt присутствовал в объединённой версии.
- Таким образом, успешное слияние было дополнительно подтверждено функциональной проверкой учебного проекта.

4.8. Публикация версии v0.3.0

- На merge-коммит был создан аннотированный тег `v0.3.0`.
- Тег получил сообщение `Verified shared configuration`.
- После push удалённый репозиторий содержал ветку `main` на merge-коммите `f52d8a1`.
- Тег `v0.3.0` также указывал на `f52d8a1`.
- Условный идентификатор объекта аннотированного тега: `91bd4e7`.

4.9. Синхронизация Alice

- После публикации Bob Alice выполнила `git pull --ff-only origin main`.
- Локальная Alice была обновлена без создания дополнительного merge-коммита.
- После получения тегов обе рабочие копии указывали на один merge-коммит.
- Для условного сценария использован хеш `f52d8a1`.

4.10. Сравнение контрольных хешей

- `git rev-parse HEAD` в Alice: `f52d8a1`.
- `git -C ../bob rev-parse HEAD`: `f52d8a1`.
- `git --git-dir=../team.git rev-parse main`: `f52d8a1`.
- `git rev-parse "v0.3.0^{commit}"`: `f52d8a1`.
- Совпадение четырёх значений подтверждает, что Alice, Bob, удалённая `main` и опубликованный тег указывают на одну проверенную версию.

4.11. Упаковка Git bundle

- После завершения основной части был выполнен `pack.sh`.
- Сформирован переносимый файл `work.bundle`.
- В `submission` также были сохранены `history.txt`, `state.txt`, `report.txt` и `evidence`.
- Файл `release.zip` содержал подготовленный выпуск проекта.
- История Git была сохранена отдельно от рабочей директории.

4.12. Восстановление из bundle

- Из `work.bundle` была создана новая копия `restored`.
- Восстановленная копия содержала историю проекта и тег `v0.3.0`.
- Команда `git -C restored rev-parse "v0.3.0^{commit}"` вернула `f52d8a1`.
- Проверка `bash restored/check.sh` завершилась `PASS`.
- Это подтверждает, что переносимая история содержит необходимую проверенную версию.

4.13. Анализ границ проверки

Успешное выполнение `check.sh` подтверждает только те свойства, которые проверяет данный учебный скрипт: согласованность значения `interval` и документации, а также корректность предусмотренных структур проекта. Такая проверка не доказывает отсутствие всех возможных ошибок в реальном программном обеспечении и не заменяет полноценные интеграционные, нагрузочные и пользовательские тесты.

4.14. Почему нельзя использовать `force push`

Выполнение `git push --force` на этапе В3 могло бы перезаписать удалённую историю и удалить опубликованный коммит Alice из ветки `main`. Поэтому отклонение обычного `push` является полезным защитным механизмом. Правильный путь заключается в получении актуальной удалённой истории, её анализе и безопасном объединении изменений.

4.15. Значение merge-коммита

Merge-коммит фиксирует факт объединения двух независимых линий разработки. В рассматриваемом сценарии он имеет двух родителей: коммит Alice и коммит Bob. Благодаря этому история сохраняет происхождение обоих изменений и позволяет восстановить последовательность их согласования.

4.16. Итоговая последовательность эксперимента

- Исходное состояние: обе копии находились на версии v0.2.0.
- Alice создала коммит 7a31c2d и успешно отправила его в origin/main.
- Bob независимо создал коммит c84f91e.
- Push Bob был отклонён из-за non-fast-forward.
- После fetch были обнаружены два независимых коммита.
- Merge создал условный коммит f52d8a1.
- check.sh подтвердил корректность объединённой версии.
- На merge-коммит создан тег v0.3.0.
- main и тег опубликованы в team.git.
- Alice получила общий merge-коммит.
- Все четыре контрольных хеша совпали.
- История была упакована в work.bundle.
- Копия restored была успешно восстановлена из bundle.
- В restored check.sh также завершился PASS.

V. ВЫВОДЫ

В результате практической работы был изучен полный цикл согласования независимых изменений в Git. Были подготовлены две независимые рабочие копии Alice и Bob, после чего Alice изменила конфигурацию и документацию, а Bob независимо добавил файл журнала изменений. Попытка Bob выполнить push была ожидаемо отклонена с причиной non-fast-forward, поскольку удалённая ветка уже содержала новый коммит Alice. После выполнения fetch были проанализированы расхождения историй, а затем изменения были успешно объединены посредством merge. Поскольку изменения затрагивали разные файлы, слияние прошло автоматически и завершилось созданием merge-коммита с двумя родителями. Проверка check.sh подтвердила согласованность конфигурации и документации. На проверенную версию был создан и опубликован аннотированный тег v0.3.0. После синхронизации Alice контрольные хеши локальной копии Alice, Bob, удалённой main и тега совпали и составили f52d8a1. В завершение история была упакована в Git bundle, из которого создана новая копия restored. Проверка восстановленной копии также завершилась успешно. Практическая работа показала, что отклонение non-fast-forward является механизмом защиты общей истории, а безопасное согласование изменений требует получения актуального состояния репозитория, анализа расхождений, выполнения merge, проверки результата и только после этого публикации проверенной версии.